Experiment 1: Generate Text Embeddings and Perform Semantic Similarity Search

Python program

```
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer("all-MiniLM-L6-v2")

sentences = [

    "AI is changing technology",

    "Machine learning is part of AI",

    "I like playing cricket",

    "Computers use AI to learn"

]

query = "Artificial Intelligence"

embeddings = model.encode(sentences, convert_to_tensor=True)

query_embedding = model.encode(query, convert_to_tensor=True)

scores = util.cos_sim(query_embedding, embeddings)[0]

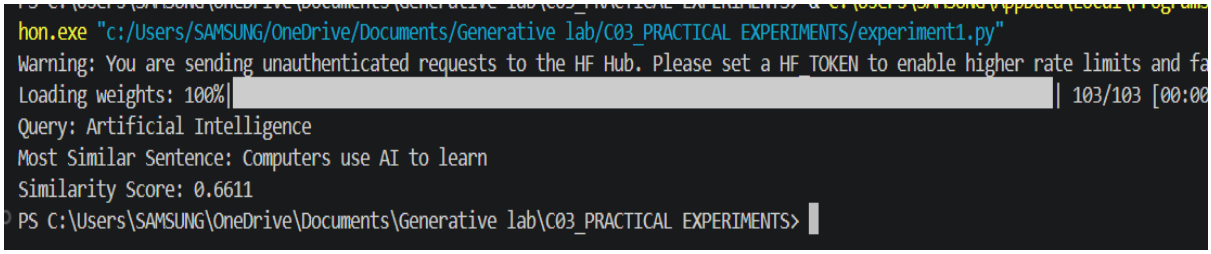
best = scores.argmax().item()

print("Query:", query)

print("Most Similar Sentence:", sentences[best])

print("Similarity Score:", round(scores[best].item(), 4))
```

Output



Experiment 2: Develop a Semantic Search System Using Cosine Similarity

Python program

```
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer("all-MiniLM-L6-v2")

documents = [
    "Artificial Intelligence makes machines smart",
    "Machine learning helps computers learn from data",
    "Python is a programming language",
    "Cricket is a popular sport"
]

query = input("Enter your query: ")

doc_embeddings = model.encode(documents, convert_to_tensor=True)

query_embedding = model.encode(query, convert_to_tensor=True)

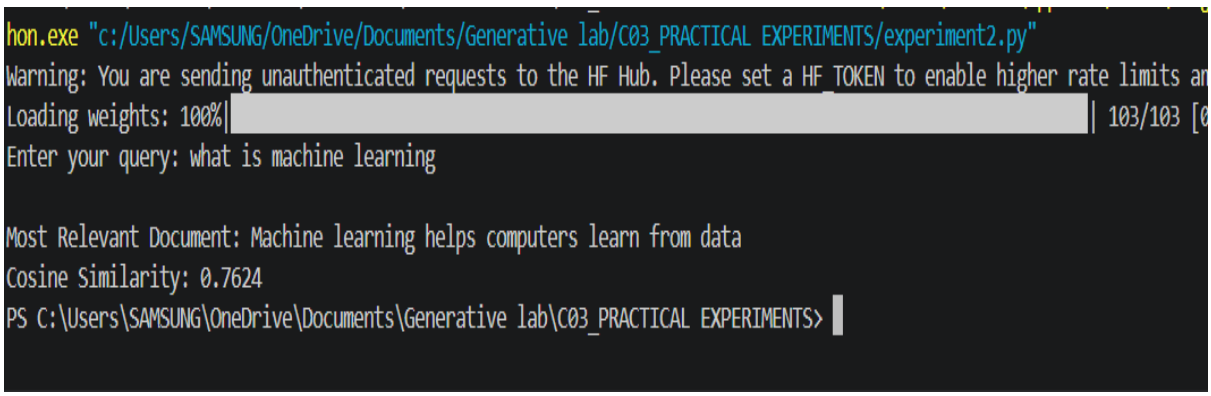
scores = util.cos_sim(query_embedding, doc_embeddings)[0]

best = scores.argmax().item()

print("\nMost Relevant Document:", documents[best])

print("Cosine Similarity:", round(scores[best].item(), 4))
```

Output



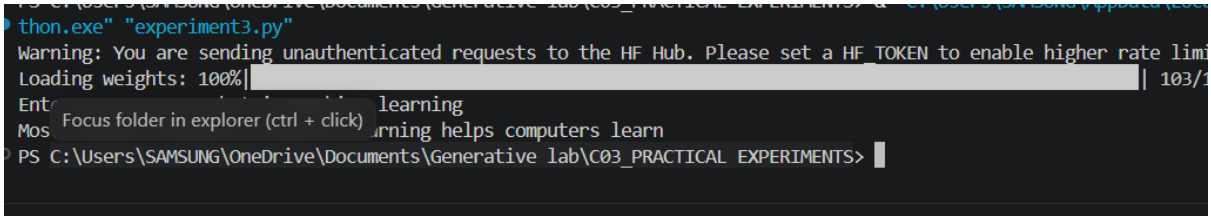
Experiment 3: Build a Vector Database Using ChromaDB

Python program

```
import chromadb
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("all-MiniLM-L6-v2")
client = chromadb.Client()
collection = client.create_collection("documents")
docs = [
    "Artificial Intelligence makes machines smart",
    "Machine learning helps computers learn",
    "Python is a programming language",
    "Cricket is a popular sport"
]
embeddings = model.encode(docs).tolist()
collection.add(
    ids=["1", "2", "3", "4"],
    documents=docs,
    embeddings=embeddings
)
query = input("Enter your query: ")
query_embedding = model.encode([query]).tolist()
result = collection.query(
    query_embeddings=query_embedding,
```

```
    n_results=1
)
print("Most Similar Document:", result["documents"][0][0])
```

Output



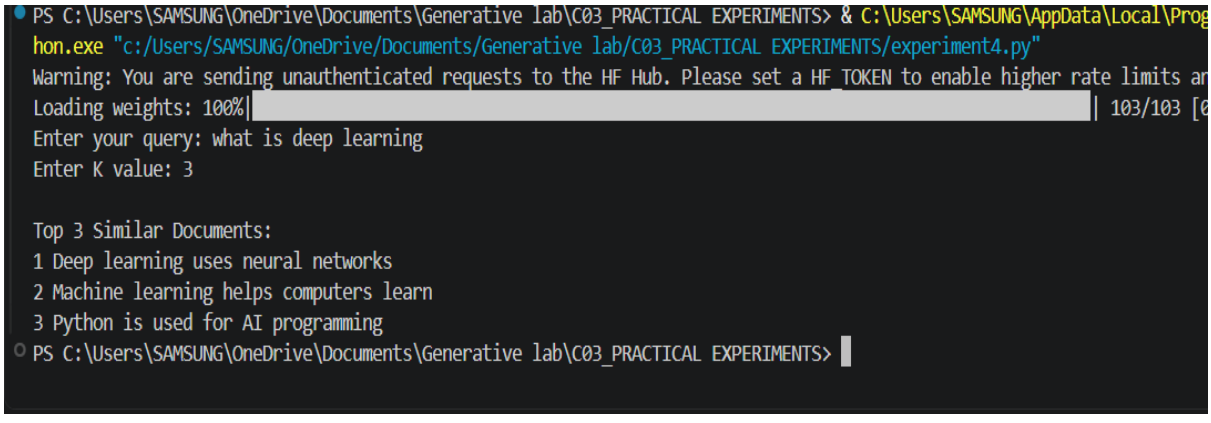
4. Implement document storage and top-k retrieval using a vector database.

Python program

```
import chromadb
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("all-MiniLM-L6-v2")
client = chromadb.PersistentClient(path="./vector_db")
collection = client.get_or_create_collection("documents")
docs = [
    "Artificial Intelligence makes machines smart",
    "Machine learning helps computers learn",
    "Python is used for AI programming",
    "Deep learning uses neural networks",
   "Cricket is a popular sport"
]
embeddings = model.encode(docs).tolist()
```

```
collection.upsert(
    ids=["1", "2", "3", "4", "5"],
    documents=docs,
    embeddings=embeddings
)
query = input("Enter your query: ")
k = int(input("Enter K value: "))
query_embedding = model.encode([query]).tolist()
result = collection.query(
    query_embeddings=query_embedding,
    n_results=min(k, len(docs))
)
print("\nTop", k, "Similar Documents:")
for i, doc in enumerate(result["documents"][0], 1):
    print(i, doc)
```

Output



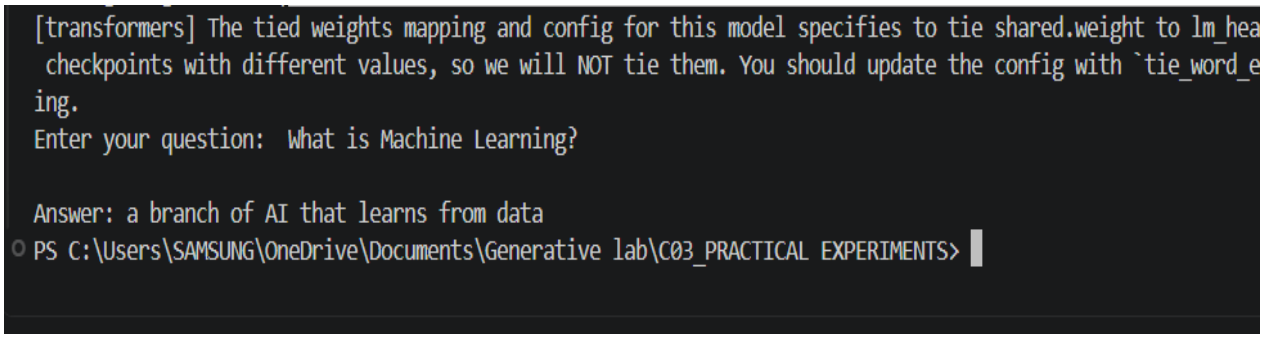
5. Develop a document Question Answering system using Retrieval-Augmented Generation (RAG).

Python program

```
import chromadb
from sentence_transformers import SentenceTransformer
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
model = SentenceTransformer("all-MiniLM-L6-v2")
client = chromadb.Client()
collection = client.get_or_create_collection("rag_docs")
docs = [
    "Artificial Intelligence makes machines intelligent.",
    "Machine Learning is a branch of AI that learns from data.",
    "Python is used for AI programming.",
    "Deep Learning uses neural networks."
]
embeddings = model.encode(docs).tolist()
collection.upsert(
    ids=["1", "2", "3", "4"],
    documents=docs,
    embeddings=embeddings
)
tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-small")
generator = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-small")
question = input("Enter your question: ")
```

```
query_embedding = model.encode([question]).tolist()
result = collection.query(
    query_embeddings=query_embedding,
    n_results=2
)
context = " ".join(result["documents"][0])
prompt = f"Answer using only the context: {context}\nQuestion: {question}"
inputs = tokenizer(prompt, return_tensors="pt", truncation=True)
outputs = generator.generate(**inputs, max_new_tokens=60)
answer = tokenizer.decode(outputs[0], skip_special_tokens=True)
print("\nAnswer:", answer)
```

Output



6. Build an end-to-end RAG pipeline involving document loading, text chunking, embeddings, retrieval, and answer generation.

Python program

```
import chromadb
from sentence_transformers import SentenceTransformer
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
```

```
text = """
Artificial Intelligence makes machines intelligent.
Machine Learning helps computers learn from data.
Deep Learning uses artificial neural networks.
Python is used for AI programming.
"""

chunks = [s.strip() for s in text.split("\n") if s.strip()]

model = SentenceTransformer("all-MiniLM-L6-v2")
embeddings = model.encode(chunks).tolist()

client = chromadb.Client()
collection = client.get_or_create_collection("rag_pipeline")

collection.upsert(
    ids=[str(i) for i in range(len(chunks))],
    documents=chunks,
    embeddings=embeddings
)

question = input("Enter your question: ")
query_embedding = model.encode([question]).tolist()

result = collection.query(
    query_embeddings=query_embedding,
    n_results=2
)

context = " ".join(result["documents"][0])
```

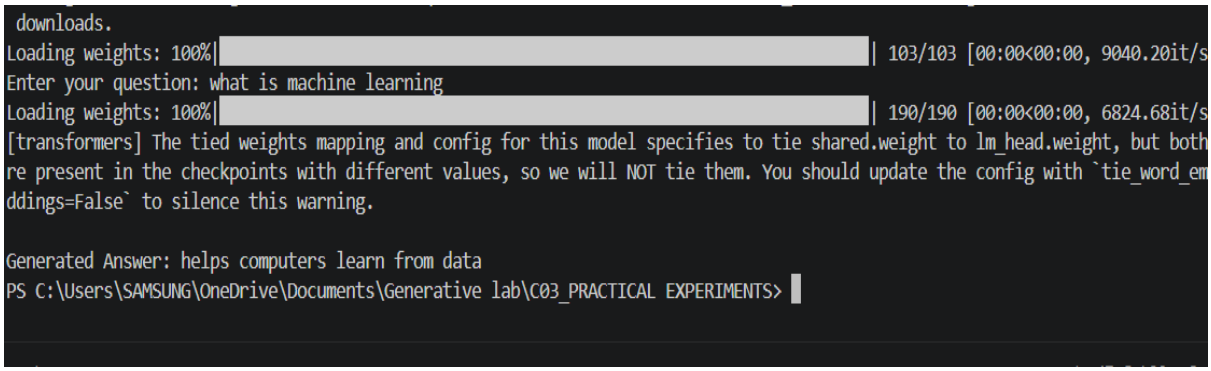
```
tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-small")
generator = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-small")

prompt = f"Answer using only the context.\nContext: {context}\nQuestion: {question}"

inputs = tokenizer(prompt, return_tensors="pt", truncation=True)
outputs = generator.generate(**inputs, max_new_tokens=60)

answer = tokenizer.decode(outputs[0], skip_special_tokens=True)
print("\nGenerated Answer:", answer)
```

Output



7. Develop a domain-specific chatbot using LangChain and a vector database.

Python program

```
from langchain_chroma import Chroma
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_core.documents import Document
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

texts = [
    "Artificial Intelligence makes machines intelligent.",
    "Machine Learning helps computers learn from data.",
```

```
    "Deep Learning uses neural networks.",
    "Python is used for AI programming."
]

docs = [Document(page_content=t) for t in texts]

embedding = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

db = Chroma.from_documents(
    documents=docs,
    embedding=embedding,
    collection_name="ai_chatbot"
)

tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-small")
model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-small")

print("AI Chatbot Ready! Type exit to stop.")

while True:
    question = input("\nYou: ")

    if question.lower() == "exit":
        break

    results = db.similarity_search(question, k=2)
    context = " ".join(doc.page_content for doc in results)

    prompt = f"Answer using only the context.\nContext: {context}\nQuestion: {question}"
```

```
    inputs = tokenizer(prompt, return_tensors="pt", truncation=True)
    outputs = model.generate(**inputs, max_new_tokens=60)

    answer = tokenizer.decode(outputs[0], skip_special_tokens=True)

    print("Chatbot:", answer)
```

Output

```
[transformers] The tied weights mapping and config for this model specifies to tie shared.weig
re present in the checkpoints with different values, so we will NOT tie them. You should updat
ddings=False` to silence this warning.
AI Chatbot Ready! Type exit to stop.

You: what is machine learning
Chatbot: helps computers learn from data

You: what is deep learning
Chatbot: Machine Learning helps computers learn from data

You: what is ARITIIFICAL LEARNING
Chatbot: Machine Learning helps computers learn from data

You: CLS
Chatbot: if python is used for AI programming then what is it?
```

8. Implement a context-aware chatbot using LangChain, retrieval, and an LLM.

Python program

```
from langchain_chroma import Chroma
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_core.documents import Document
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
texts = [
    "Artificial Intelligence enables machines to perform intelligent tasks.",
    "Machine Learning is a branch of AI that learns from data.",
    "Deep Learning is a branch of Machine Learning that uses neural networks.",
    "Python is widely used for AI and Machine Learning."
]
```

```
docs = [Document(page_content=t) for t in texts]

embedding = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

db = Chroma.from_documents(
    documents=docs,
    embedding=embedding,
    collection_name="context_chatbot"
)

tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-small")
model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-small")

history = []
print("Context-Aware Chatbot Ready! Type exit to stop.")

while True:
    question = input("\nYou: ").strip()

    if question.lower() == "exit":
        break

    if not question:
        continue

    previous = "\n".join(history[-4:])
    search_query = previous + "\n" + question

    results = db.similarity_search(search_query, k=2)
```

```
context = " ".join(doc.page_content for doc in results)

prompt = (
    "Answer using only the context. "
    "Use chat history to understand follow-up questions.\n"
    f"Context: {context}\n"
    f"Chat History: {previous}\n"
    f"Question: {question}\nAnswer:"
)

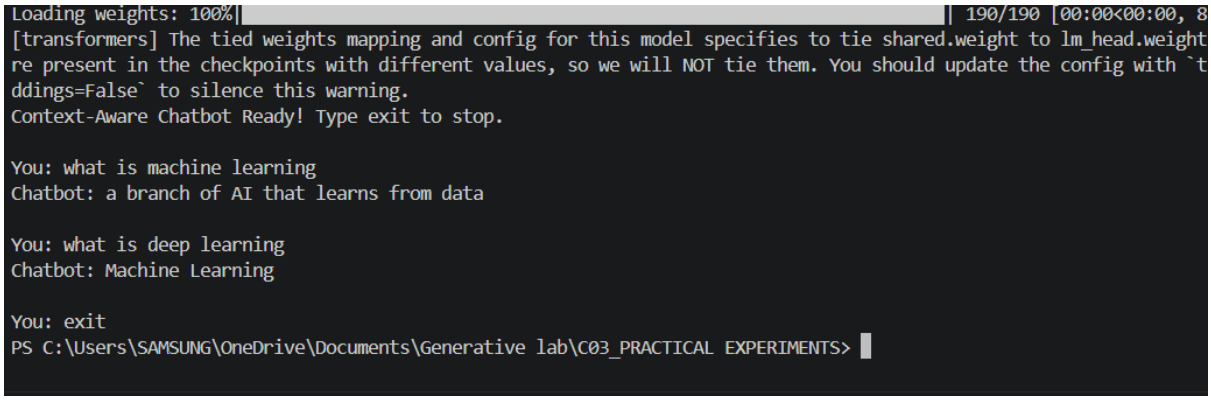
inputs = tokenizer(
    prompt, return_tensors="pt",
    truncation=True, max_length=512
)

outputs = model.generate(
    **inputs, max_new_tokens=60,
    do_sample=False
)

answer = tokenizer.decode(
    outputs[0], skip_special_tokens=True
)

print("Chatbot:", answer)
history.append("User: " + question)
history.append("Assistant: " + answer)
```

Output



Experiment 9: Develop a Simple AI Assistant Using External Documents

Python program

```
from pathlib import Path
import chromadb
from sentence_transformers import SentenceTransformer
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

file = Path(__file__).parent / "document.txt"

if not file.exists():
    file.write_text(
        "Artificial Intelligence makes machines intelligent.\n"
        "Machine Learning helps computers learn from data.\n"
        "Deep Learning uses neural networks.\n"
        "Python is used for AI programming.",
        encoding="utf-8"
    )

text = file.read_text(encoding="utf-8")
docs = [line.strip() for line in text.splitlines() if line.strip()]
```

```
embedder = SentenceTransformer("all-MiniLM-L6-v2")
embeddings = embedder.encode(docs).tolist()

client = chromadb.Client()
db = client.get_or_create_collection("assistant_docs")

db.upsert(
    ids=[str(i) for i in range(len(docs))],
    documents=docs,
    embeddings=embeddings
)

tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-small")
model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-small")

question = input("Enter your question: ")
query = embedder.encode([question]).tolist()

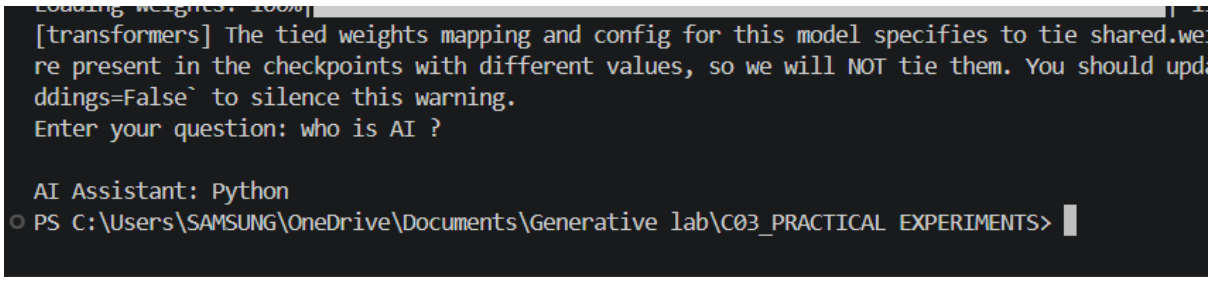
result = db.query(
    query_embeddings=query,
    n_results=min(2, len(docs))
)

context = " ".join(result["documents"][0])
prompt = f"Answer using only the context.\nContext: {context}\nQuestion: {question}"

inputs = tokenizer(prompt, return_tensors="pt", truncation=True)
outputs = model.generate(**inputs, max_new_tokens=60)

answer = tokenizer.decode(outputs[0], skip_special_tokens=True)
print("\nAI Assistant:", answer)
```

Output



10. Build a document-based AI assistant that supports multiple uploaded documents and question answering.

Python program

```
from pathlib import Path
import chromadb
from sentence_transformers import SentenceTransformer
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

folder = Path(__file__).parent / "documents"
folder.mkdir(exist_ok=True)

files = list(folder.glob("*.txt"))

if not files:
    (folder / "ai.txt").write_text(
        "Artificial Intelligence makes machines intelligent.",
        encoding="utf-8"
    )
    (folder / "ml.txt").write_text(
        "Machine Learning helps computers learn from data.",
        encoding="utf-8"
    )
    files = list(folder.glob("*.txt"))
```

```
chunks = []
for file in files:
    text = file.read_text(encoding="utf-8")
    for i in range(0, len(text), 300):
        chunk = text[i:i+300].strip()
        if chunk:
            chunks.append(chunk)

embedder = SentenceTransformer("all-MiniLM-L6-v2")
embeddings = embedder.encode(chunks).tolist()

client = chromadb.Client()
db = client.get_or_create_collection("multi_docs")

db.upsert(
    ids=[str(i) for i in range(len(chunks))],
    documents=chunks,
    embeddings=embeddings
)

tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-small")
model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-small")

print("AI Assistant Ready! Type exit to stop.")

while True:
    question = input("\nYou: ").strip()

    if question.lower() == "exit":
        break
    if not question:
```

```
    continue

query = embedder.encode([question]).tolist()

result = db.query(
    query_embeddings=query,
    n_results=min(2, len(chunks))
)

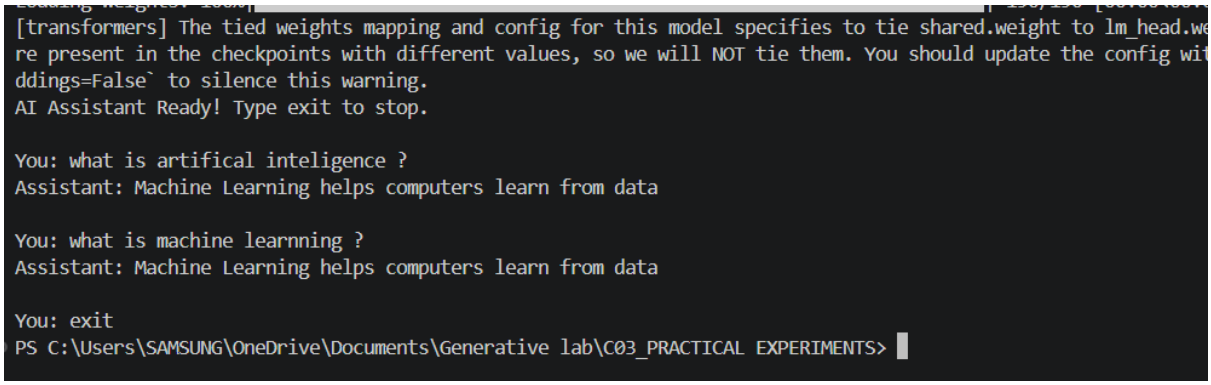
context = " ".join(result["documents"][0])

prompt = (
    f"Answer using only the context.\n"
    f"Context: {context}\n"
    f"Question: {question}\nAnswer:"
)

inputs = tokenizer(
    prompt, return_tensors="pt",
    truncation=True, max_length=512
)

outputs = model.generate(
    **inputs, max_new_tokens=60,
    do_sample=False
)
answer = tokenizer.decode(
    outputs[0], skip_special_tokens=True
)
print("Assistant:", answer)
```

Output


[transformers] The tied weights mapping and config for this model specifies to tie shared.weight to lm_head.we
re present in the checkpoints with different values, so we will NOT tie them. You should update the config wit
ddings=False` to silence this warning.
AI Assistant Ready! Type exit to stop.
You: what is artifical inteligence ?
Assistant: Machine Learning helps computers learn from data
You: what is machine learnning ?
Assistant: Machine Learning helps computers learn from data
You: exit
PS C:\Users\SAMSUNG\OneDrive\Documents\Generative lab\C03_PRACTICAL EXPERIMENTS>